

Управление памятью

Типы значений и ссылочный типы

Все типы данных можно разделить на типы значений, еще называемые значимыми типами, (value types) и ссылочные типы (reference types).

Типы значений

- Целочисленные типы (`byte`, `sbyte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`)
- Типы с плавающей запятой (`float`, `double`)
- Тип `decimal`
- Тип `bool`
- Тип `char`
- Перечисления `enum`
- Структуры (`struct`)

Ссылочные типы

- Тип `object`
- Тип `string`
- Классы (`class`)
- Интерфейсы (`interface`)
- Делегаты (`delegate`)

В чем разница

Память делится на два типа: стек и куча (heap).

Стек

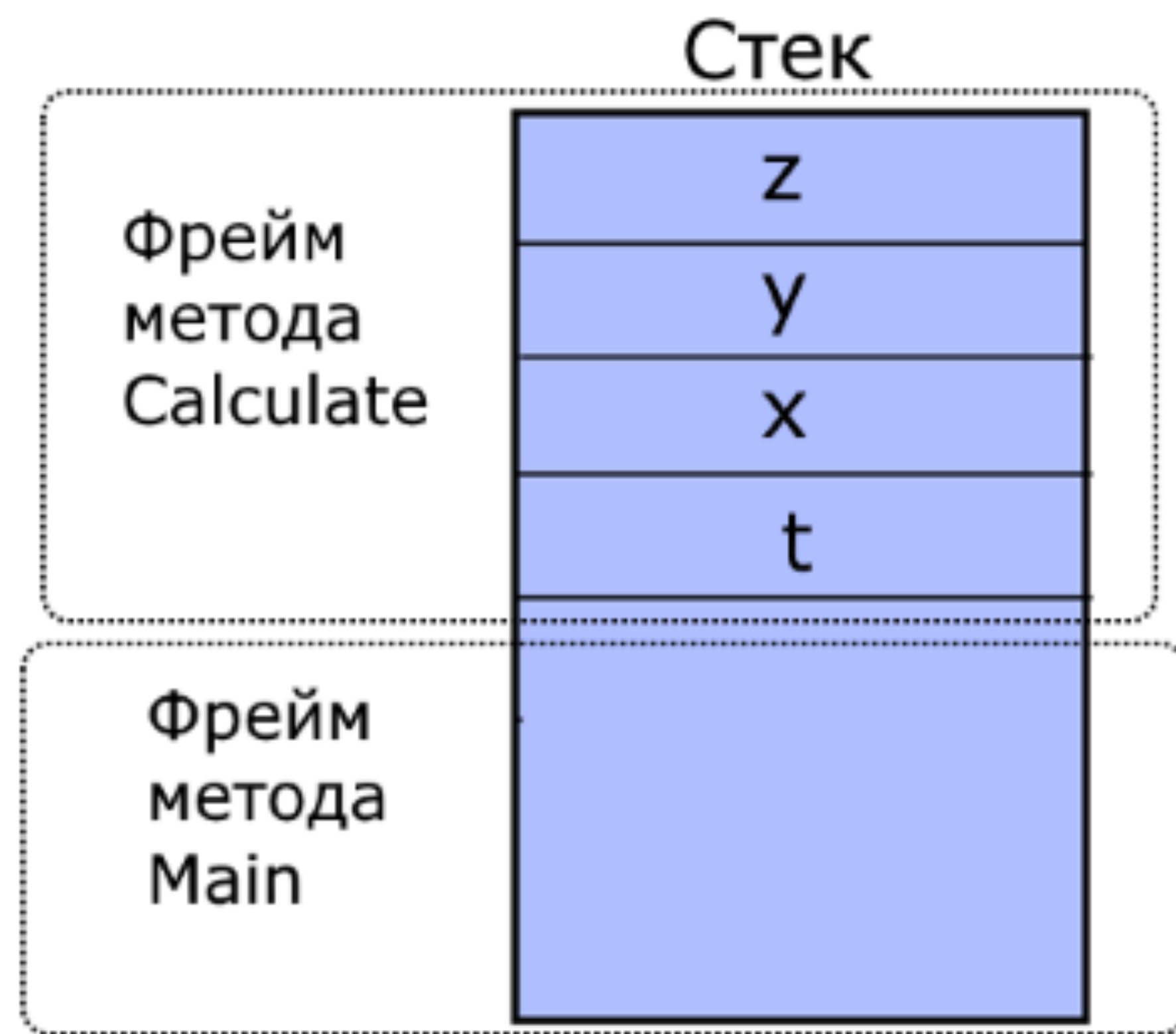
Параметры и переменные метода, которые представляют типы значений, размещают свое значение в стеке. Стек представляет собой структуру данных, которая растет снизу вверх: каждый новый добавляемый элемент помещается поверх предыдущего. Время жизни переменных таких типов ограничено их контекстом. Физически стек - это некоторая область памяти в адресном пространстве.

Пример

```
class Program
{
    static void Main(string[] args)
    {
        Calculate(5);
    }

    static void Calculate(int t)
    {
        int x = 6;
        int y = 7;
        int z = y + t;
    }
}
```

Пример



Пример

При вызове этого метода Calculate в его фрейм в стеке будут помещаться значения t , x , y и z

Когда метод отработает, область памяти, которая выделялась под стек, впоследствии может быть использована другими методами.

Причем если параметр или переменная метода представляет тип значений, то в стеке будет храниться непосредственное значение этого параметра или переменной.

Ссылочные типы

Ссылочные типы хранятся в куче, которую можно представить как неупорядоченный набор разнородных объектов. Физически это оставшаяся часть памяти, которая доступна процессу.

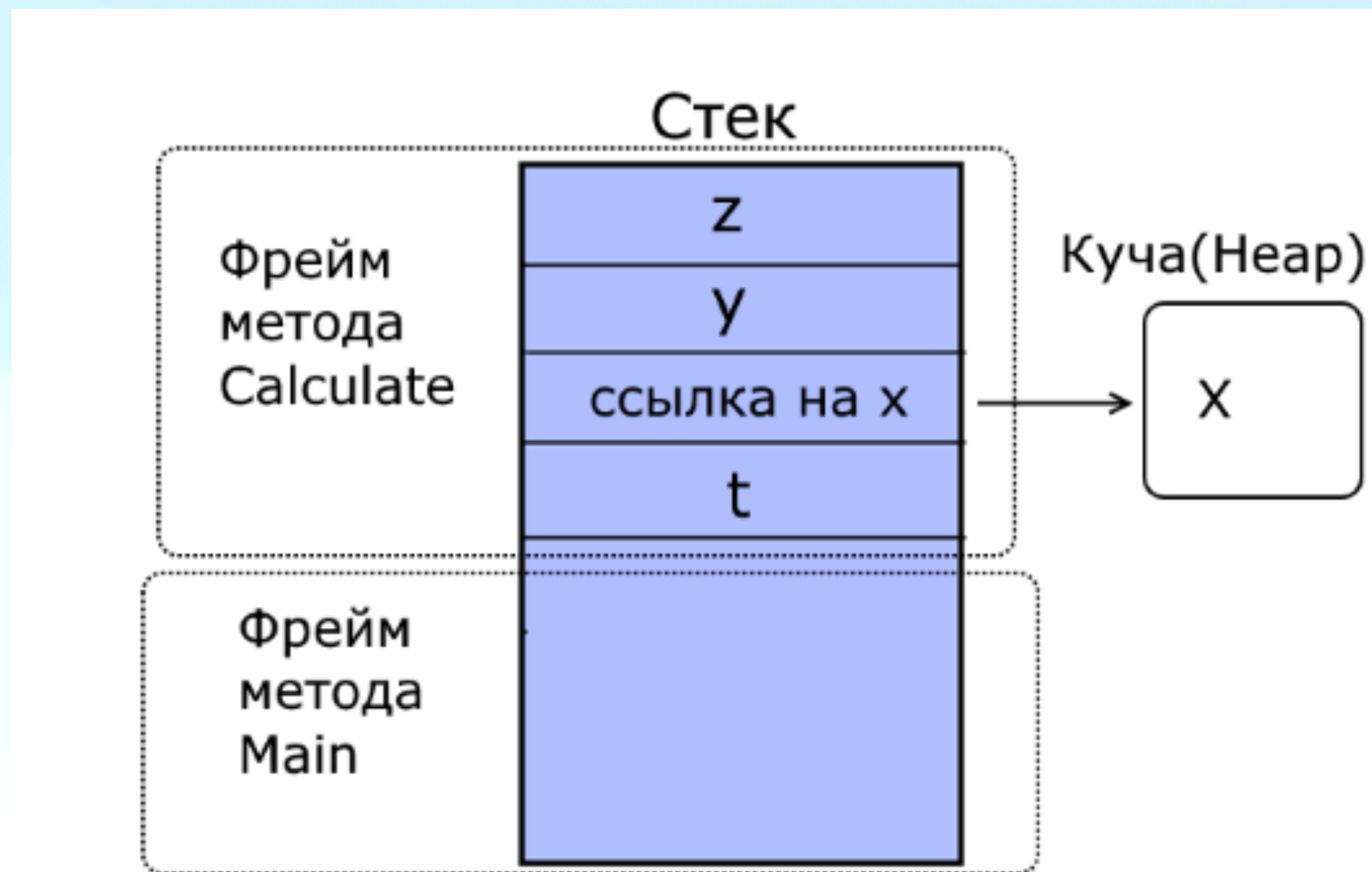
Ссылочные типы

При создании объекта ссылочного типа в стеке помещается ссылка на адрес в куче. Когда объект ссылочного типа перестает использоваться, в дело вступает автоматический сборщик мусора

Пример2

```
static void Calculate(int t)
{
    object x = 6;
    int y = 7;
    int z = y + t;
}
```

Пример2

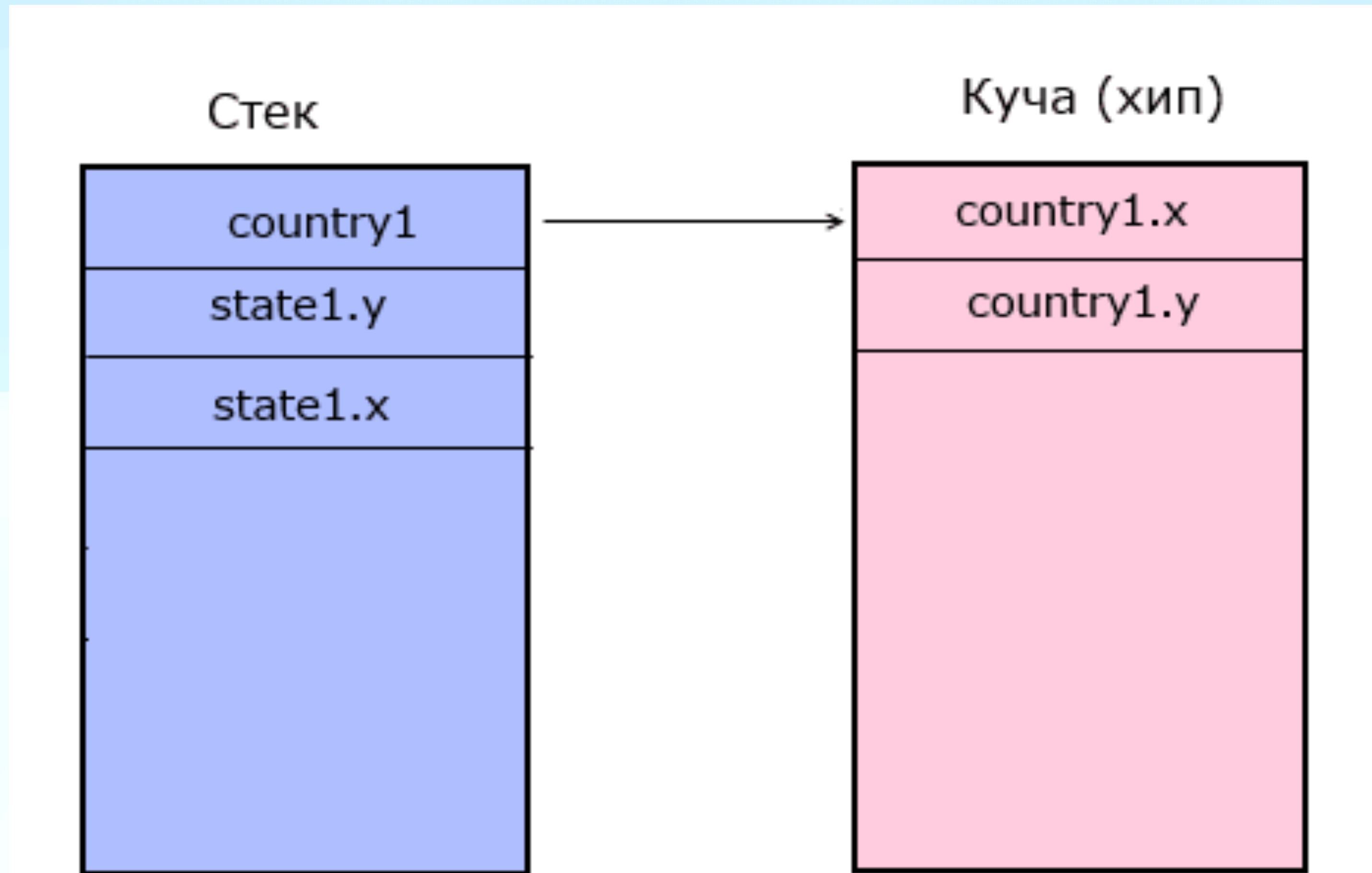


Составные типы

```
State state1 = new State(); // State – структура, ее данные размещены в стеке
Country country1 = new Country(); // Country – класс, в стек помещается ссылка
на адрес в хипе // а в хипе располагаются все данные объекта
```

```
country1
struct State
{
    public int x;
    public int y;
}
class Country
{
    public int x;
    public int y;
}
```

Составные типы

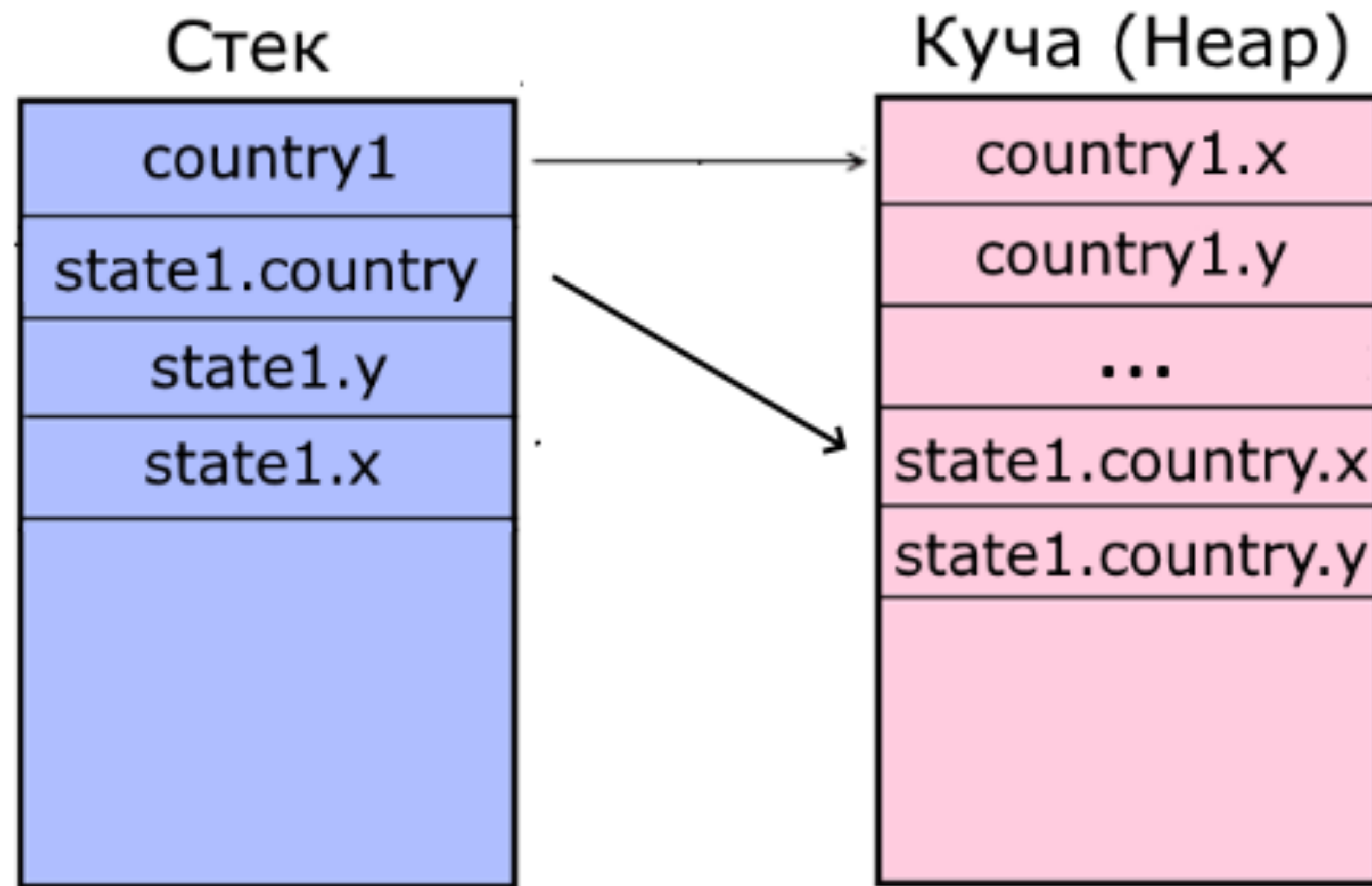


Пример3

```
State state1 = new State();  
Country country1 = new Country();
```

```
struct State  
{  
    public int x;  
    public int y;  
    public Country country;  
    public State()  
    {  
        x = 0;  
        y = 0;  
        country = new Country();  
    }  
}  
class Country  
{  
    public int x;  
    public int y;  
}
```

Составные типы



Копирование значений

Тип данных надо учитывать при копировании значений. При присвоении данных объекту значимого типа он получает копию данных. При присвоении данных объекту ссылочного типа он получает не копию объекта, а ссылку на этот объект в хипе.

Пример4

```
State state1 = new State(); // Структура State
State state2 = new State();
state2.x = 1;
state2.y = 2;
state1 = state2;
state2.x = 5; // state1.x=1 по-прежнему
Console.WriteLine(state1.x); // 1
Console.WriteLine(state2.x); // 5
```

```
Country country1 = new Country(); // Класс Country
Country country2 = new Country();
country2.x = 1;
country2.y = 4;
country1 = country2;
country2.x = 7; // теперь и country1.x = 7, так как обе ссылки и country1 и country2
                // указывают на один объект в хипе
Console.WriteLine(country1.x); // 7
Console.WriteLine(country2.x); // 7
```

Сборщик мусора

Переменные типов

При использовании переменных типов значений в методе, все значения этих переменных попадают в стек. После завершения работы метода стек очищается.

Составные типы

При использовании же ссылочных типов, для них также будет отводиться место в стеке, только там будет храниться не значение, а адрес на участок памяти в хипе или куче, в котором и будут находиться сами значения данного объекта. И если объект класса перестает использоваться, то при очистке стека ссылка на участок памяти также очищается, однако это не приводит к немедленной очистке самого участка памяти в куче. Впоследствии сборщик мусора (garbage collector) увидит, что на данный участок памяти больше нет ссылок, и очистит его.

Пример5

```
Test();
```

```
void Test()  
{  
    Person tom = new Person("Tom");  
    Console.WriteLine(tom.Name);  
}  
record class Person(string Name);
```

Сборщик мусора

Как правило, объекты в куче располагаются неупорядочено, между ними могут иметься пустоты. Куча довольно сильно фрагментирована. Поэтому после очистки памяти в результате очередной сборки мусора оставшиеся объекты перемещаются в один непрерывный блок памяти. Вместе с этим происходит обновление ссылок, чтобы они правильно указывали на новые адреса объектов.

Сборщик мусора

Для крупных объектов существует своя куча - **Large Object Heap**.
В эту кучу помещаются объекты, размер которых больше 85 000 байт.

Сборщик мусора

Чтобы снизить издержки от работы сборщика мусора, все объекты в куче разделяются по поколениям. Всего существует три поколения объектов: 0, 1 и 2-е.

К поколению 0 относятся новые объекты, которые еще ни разу не подвергались сборке мусора. К поколению 1 относятся объекты, которые пережили одну сборку, а к поколению 2 - объекты, прошедшие более одной сборки мусора.

Сборщик мусора

Когда сборщик мусора приступает к работе, он сначала анализирует объекты из поколения 0. Те объекты, которые остаются актуальными после очистки, повышаются до поколения 1.

Если после обработки объектов поколения 0 все еще необходима дополнительная память, то сборщик мусора приступает к объектам из поколения 1. Те объекты, на которые уже нет ссылок, уничтожаются, а те, которые по-прежнему актуальны, повышаются до поколения 2.

Класс **System.GC**

Функционал сборщика мусора в библиотеке классов .NET представляет класс **System.GC**. Через статические методы данный класс позволяет обращаться к сборщику мусора.

Класс System.GC

Методы класса

- AddMemoryPressure
- Collect
- GetGeneration(Object)
- GetTotalMemory
- WaitForPendingFinalizers